

ค้นหาลำดับ

- ชีตจำกัดเวลา: 10 minutes
- สภาพแวดล้อม: GPU หนึ่งตัว (≈ 16 GB VRAM), ไม่มีอินเทอร์เน็ต
- ขนาดโซลูชัน: `solution.ipynb` ≤ 1 MB
- พื้นที่จัดเก็บ: 5 GB

โจทย์

คุณได้รับบทสนทนาภาษาอังกฤษแบบพูดระหว่างผู้เข้าร่วมสองคน ได้แก่ ผู้พูด A และ ผู้พูด B บทสนทนาแต่ละรายการถูกแบ่งเป็นผลัดการพูด (speaker turn) โดยแต่ละผลัดมีคำพูดจากผู้พูดเพียงคนเดียว แต่ละผลัดถูกจัดเก็บเป็นไฟล์เสียง .wav แยกกัน ดังนั้นบทสนทนาที่สมบูรณ์จึงแสดงแทนด้วยชุดไฟล์ .wav โดยมีหนึ่งไฟล์ต่อหนึ่งผลัดการพูด

น่าเสียดายที่ผลัดการพูดถูกสับแบบสุ่ม ทำให้บทสนทนาไม่สมเหตุสมผลอีกต่อไป ในชื่อไฟล์ `chunk_{k}.wav` นั้น k หมายถึงชิ้นส่วน (chunk) ลำดับที่ k ในชุดที่ถูกสับ ไม่ใช่ผลัดการพูดลำดับที่ k ในบทสนทนาดั้งเดิม

!! งานของคุณคือสร้างลำดับตามเวลาของบทสนทนาดั้งเดิมขึ้นใหม่



ชุดข้อมูล

บทสนทนาแต่ละรายการประกอบด้วยไฟล์เสียง n ซึ่งมีชื่อว่า `chunk_0.wav`, `chunk_1.wav`, ..., `chunk_{n-1}.wav` ชิ้นส่วนเหล่านี้คือผลการพูดแต่ละผลัด ชื่อไฟล์สอดคล้องกับลำดับที่ถูกสลับเท่านั้น และไม่ได้บ่งชี้ว่าชิ้นส่วนหนึ่งอยู่ตำแหน่งใดในบทสนทนาดั้งเดิม แต่ละบทสนทนามีชิ้นส่วน 7-20 ชิ้น เป็นเสียงโมโน 44.1 kHz (คุณอาจ ปรับอัตรา การสุ่มตัวอย่างใหม่ได้)

`prefix.json` มีดัชนีชื่อไฟล์ของชิ้นส่วนสองชิ้นแรกในแต่ละบทสนทนา ข้อมูลนี้ระบุจุดเริ่มต้นที่แท้จริงของบทสนทนา และจัดความกำกวมระหว่างการอ่านบทสนทนาไปข้างหน้าหรือย้อนกลับ

ตัวอย่างเช่น 11: [7, 12] หมายความว่าผลการพูดแรกและผลการพูดที่สองของบทสนทนา 11 คือ `chunk_7.wav` และ `chunk_12.wav` ตามลำดับ

สิ่งที่คุณได้รับ

คุณจะได้รับ สองไฟล์เดอร์ที่มีรูปแบบเหมือนกัน:

โฟลเดอร์	บทสนทนา	<code>answers.json</code> ?	ใช้เพื่อ
<code>dataset/train/</code>	1,288	✓ มีให้	ฝึก / fine-tune โมเดล (model) ของคุณ
<code>dataset/test_public/</code>	100	✓ มีให้	รัน pipeline และคำนวณคะแนนของคุณเองภายในเครื่อง

ในช่วงตรวจให้คะแนน โฟลเดอร์ `dataset/test_public/` ของคุณจะถูกแทนที่โดยอัตโนมัติด้วย hidden evaluation set (`test_leaderboard_a` สำหรับ leaderboard สาธารณะ และ `test_leaderboard_b` สำหรับ leaderboard รอบสุดท้าย) ซึ่งมีขนาดและรูปแบบเดียวกับ `dataset/test_public/` แต่ไม่มี `answers.json`

จากนั้น notebook ของคุณจะถูกเรียกใช้อีกครั้งกับข้อมูลดังกล่าว และไฟล์ `answers.json` ที่สร้างขึ้นจะถูกนำไปใช้คำนวณคะแนน บทสนทนาทดสอบที่กันไว้มาจากการแจกแจงเดียวกันกับ `train` ดังนั้นคะแนน `test_public` ภายในเครื่องของคุณจึงเป็นตัวอย่างล่วงหน้าที่น่าเชื่อถือ

โครงสร้างไดเรกทอรี

```
dataset/train/
  prefix.json # {dialogue_id: [first_idx, second_idx]} filename index
  answers.json # {dialogue_id: P} ground-truth order (rank convention)
  /
    chunk_0.wav
    ...
    chunk_{n-1}.wav

dataset/test_public/
  prefix.json
  answers.json # present only in the development copy
  /
    chunk_0.wav
    ...
    chunk_{n-1}.wav
```

ผลลัพธ์

สำหรับแต่ละบทสนทนา ให้หาลำดับตามเวลาดั้งเดิมของชิ้นส่วนเสียง การทำนายของคุณควรเป็นการเรียงสับเปลี่ยน (permutation) P ของ $\{0, 1, \dots, n-1\}$ โดยที่ $P[i]$ คือตำแหน่งตามเวลาที่ทำนายไว้ของ `chunk_i.wav` ($0 =$ ลำดับแรก)

ไฟล์ผลลัพธ์ `answers.json` ของคุณควรจับคู่ ID ของแต่ละบทสนทนากับการเรียงสับเปลี่ยนที่ทำนายไว้:

```
{
  "17": [2, 0, 1],
  "42": [0, 1],
  "108": [3, 1, 0, 4, 2, 5]
}
```

ตัวอย่าง

บทสนทนาหนึ่งมีชิ้นส่วนที่ถูกสับ 3 ชิ้น `chunk_0`, `chunk_1`, `chunk_2`:

ชิ้นส่วนที่ถูกสับ	เนื้อหาคำพูด	ตำแหน่งจริง (อันดับ)
<code>chunk_0.wav</code>	"No worries — I'll send you the notes afterwards."	2 (สุดท้าย)
<code>chunk_1.wav</code>	"Hey, are you coming to the three o'clock meeting?"	0 (แรก)
<code>chunk_2.wav</code>	"I can't — I've got a dentist appointment then."	1

ลำดับจริงคือ **chunk_1** → **chunk_2** → **chunk_0** ดังนั้น $P = [2, 0, 1]$ และ `prefix.json` เก็บค่า $[1, 2]$

⚠ **P ต้องเป็นการเรียงสับเปลี่ยนที่ถูกต้อง:** มีความยาว n , ใช้ดัชนีเริ่มที่ 0, และแต่ละค่าปรากฏเพียงครั้งเดียว ค่าซ้ำ ค่าที่หายไป หรือค่าที่อยู่นอกช่วง (เช่น ใช้ดัชนีเริ่มที่ 1) จะได้คะแนน 0 สำหรับบทสนทนานั้น เช่นเดียวกับกรณีที่บทสนทนาหายไปจากไฟล์ ไฟล์ที่มีรูปแบบไม่ถูกต้องหรือไม่ใช่ JSON จะถูกปฏิเสธ

การให้คะแนน

การให้คะแนนสำหรับงานนี้ใช้ **ความแม่นยำของลำดับแบบเป็นคู่ (pairwise ordering accuracy)** โดยจะตรวจสอบชิ้นส่วนทุกคู่และถามว่า: *ชิ้นใดในสองชิ้นควรมาก่อน* คู่หนึ่งถือว่าถูกต้องหากการทำนายของคุณให้คำตอบเดียวกับค่าจริง (ground truth) สำหรับบทสนทนาที่มีชิ้นส่วน n ชิ้น จะมีคู่ทั้งหมด

$$M = n(n - 1)/2$$

คู่ ให้ I เป็นจำนวนอินเวอร์ชัน (inversion) — คู่ที่เรียงลำดับแตกต่างจากค่าจริง:

$$\text{score} = 1 - \frac{I}{M} \in [0, 1]$$

i คะแนนสุดท้ายคือค่าเฉลี่ยของคะแนนรายบทสนทนาจากบทสนทนาทั้งหมด ใน `split`

โมเดลที่อนุญาต

คุณสามารถใช้ได้เฉพาะโมเดลที่ฝึกล่วงหน้าต่อไปนี้เพื่อแก้ไขข้อบกพร่องนี้ ทั้งในระหว่างการฝึกและการประเมินผล โมเดลทั้งหมดนี้ได้รับการดาวน์โหลดไว้แล้วและพร้อมใช้งานในสภาพแวดล้อม คุณสามารถดูตัวอย่างวิธีใช้งานได้ใน `baseline notebook solution.ipynb` โปรดทราบว่าคุณไม่สามารถใช้โมเดลอื่นใดได้ และโปรแกรมของคุณไม่มีสิทธิ์เข้าถึงอินเทอร์เน็ต

- **ตัวแทนข้อมูลเสียงพูด (speech representation):** **wav2vec 2.0** นอกจากนี้ยังอาจใช้ **Whisper encoder** เป็นตัวสกัดฟีเจอร์ (feature extractor) ได้ [การ์ดโมเดล wav2vec](#)
- **การรู้จำเสียงพูดอัตโนมัติ (ASR):** **OpenAI Whisper** (ขนาดใดก็ได้) [การ์ดโมเดล Whisper](#)
- **โมเดลภาษา: Qwen2.5-0.5B** ซึ่งอาจใช้แบบ zero-shot หรือ fine-tune uu split train ที่ให้มา [การ์ดโมเดล Qwen](#) โปรดทราบว่าขีดจำกัด 10-minute ต้องครอบคลุมทั้งการฝึกหรือ fine-tuning ใด ๆ ที่คุณทำในช่วงตรวจให้คะแนน รวมถึงการอนุมาน (inference) บนชุดประเมินผล

วิธีส่งคำตอบ

- เปิด `solution.ipynb` และรับทุกเซลล์ ตรวจสอบว่า `notebook` เขียนไฟล์ `answers.json` ในไดเรกทอรีทำงาน โดยมีการเรียงสับเปลี่ยนสำหรับทุกบทสนทนาใน `dataset/test_public/` (100 บทสนทนา) ในช่วงตรวจให้คะแนน `notebook` จะถูกรันอีกครั้งบนชุดทดสอบที่ซ่อนไว้ และไฟล์ `answers.json` ที่สร้างขึ้นในครั้งนั้นจะถูกนำไปคำนวณคะแนน

- ปรับปรุงโซลูชันหากคุณต้องการ — หรือไม่ต้องปรับปรุงก็ได้; baseline เพียงอย่างเดียวก็ใช้ตรวจสอบความถูกต้องของ pipeline ได้
- เปิดเก็บ Git ในแถบด้านซ้ายของ JupyterLab
- **Stage** `solution.ipynb` (ไอคอน + ที่อยู่ข้างไฟล์)
- ป้อนข้อความ commit แล้วคลิก **Commit**
- คลิกไอคอนรูปเมฆที่มีลูกศรชี้ขึ้นเพื่อ push
- กลับไปยังหน้าการแข่งขันนี้แล้วคลิก **Submit**

ส่งไฟล์เพียงหนึ่งไฟล์เท่านั้น โดยตั้งชื่อว่า `solution.ipynb`